

Summary

Fundamentals of Number Systems and Bitwise Operators in C

Executive Summary

This document presents a structured technical summary of number systems and bitwise operators as applied in C programming. These concepts form the foundation for machine-level data representation, manipulation, and optimization.

The lecture emphasizes:

- Positional logic across number systems
- Binary properties such as parity
- Efficient computation using bitwise operators
- Mathematical equivalence between bitwise shifts and powers of two
- Unique properties of the XOR operator

A strong understanding of these topics is essential for low-level programming and performance-oriented problem solving.

Key Takeaways

- **Positional Logic:**
In any number system, appending a symbol to the right multiplies the existing value by the base and adds the new symbol.
- **Binary Parity Rule:**
 - Binary numbers ending in **0** are **Even**
 - Binary numbers ending in **1** are **Odd**
- **Bitwise Efficiency:**
 - Left shift ($\ll$) by k positions equals multiplication by 2^k
 - Right shift ($\gg$) by k positions equals integer division by 2^k
- **XOR Properties:**
 - $A \oplus A = 0$
 - $0 \oplus A = A$
 - XORing an odd number of times returns the number
 - XORing an even number of times returns 0

1. Overview of Number Systems

In C programming and computer science, four primary number systems are commonly used. Each system is defined by its **base** and allowable symbols.

Number System	Base	Symbols
Decimal	10	0–9
Binary	2	0, 1
Octal	8	0–7
Hexadecimal	16	0–9, A–F

1.1 Logic of Positional Value

The value of a number is determined by:

- **Face Value** (the digit itself)
- **Place Value** (base raised to the position power)

Example (Decimal System)

For the number **26**:

$$(2 \times 10^1) + (6 \times 10^0) = 20 + 6$$

Appending a Symbol

When a symbol is appended to the right:

$$\text{New Value} = (\text{Old Value} \times \text{Base}) + \text{New Symbol}$$

Example:

$$264 = (26 \times 10) + 4$$

This positional logic applies universally across all number systems.

2. Binary Conversions

Binary is the most critical number system in computing because machines operate using **0s and 1s**.

2.1 Decimal to Binary Conversion

The **repeated division-by-2 method** is used:

1. Divide the number by 2
2. Record the remainder (0 or 1)
3. Continue until quotient becomes 0
4. Read remainders from bottom to top

Example: Converting 69 to Binary

Division	Quotient	Remainder
$69 \div 2$	34	1
$34 \div 2$	17	0
$17 \div 2$	8	1
$8 \div 2$	4	0
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1

Binary Result:

1000101

2.2 Binary to Decimal Conversion

Each bit is multiplied by 2^{position} , starting from position 0 on the right.

Example: Converting 100110 to Decimal

$$(1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$$

$$32 + 0 + 0 + 4 + 2 + 0 = 38$$

3. Binary Mathematical Shortcuts

Binary patterns enable fast mental calculations:

Appending 0:

Value = $2 \times x$

-

Appending 1:

Value = $(2 \times x) + 1$

-

- **Parity Check:**

- Last bit = 0 → Even
- Last bit = 1 → Odd

4. Bitwise Operators in C

Bitwise operators work on individual bits, not entire numerical values.

4.1 Bitwise AND (&)

Result is 1 only if both bits are 1.

Truth Table

$0 \& 0 = 0$

$0 \& 1 = 0$

$1 \& 0 = 0$

$1 \& 1 = 1$

Example: 9 & 13

9 = 1001

13 = 1101

1001 (Decimal 9)

4.2 Bitwise OR (|)

Result is 1 if at least one bit is 1.

Truth Table

$0 | 0 = 0$

0 | 1 = 1
1 | 0 = 1
1 | 1 = 1

Example: 6 | 10

6 = 0110
10 = 1010

1110 (Decimal 14)

4.3 Bitwise XOR (^)

Result is 1 if bits are different.

Truth Table

$0 \wedge 0 = 0$
 $1 \wedge 1 = 0$
 $0 \wedge 1 = 1$
 $1 \wedge 0 = 1$

Key Identities

$A \oplus A = 0$
 $0 \oplus A = A$

- XOR odd times $\rightarrow A$
- XOR even times $\rightarrow 0$

4.4 Bitwise Left Shift (<<)

Shifts bits left and fills rightmost positions with 0.

Mathematical Rule

$n \ll k = n \times 2^k$

Examples

$5 \ll 1 = 10$
 $5 \ll 2 = 20$

4.5 Bitwise Right Shift (>>)

Shifts bits right, discarding rightmost bits.

Mathematical Rule

$$n \gg k = \lfloor n / 2^k \rfloor$$

Examples

$$10 \gg 1 = 5$$

$$10 \gg 2 = 2$$

(Fractional parts are discarded.)

5. Conclusion

Bitwise operators are essential tools for binary manipulation, enabling efficient computation through masking, shifting, and parity checks. Mastery of number systems and bitwise logic is fundamental for strong proficiency in C programming and competitive examinations such as GATE. Regular practice with binary conversions and bitwise operations is necessary to build speed and accuracy.